

Università degli Studi di Torino

Dipartimento di informatica



Tesi di Laurea Triennale in Informatica

Benchmarking Time Series Classification Methods

Relatore:
Prof. Marco Aldinucci

Candidato:
Mattia Liberatore

Correlatore:
Dott. Bruno Casella

Abstract

La classificazione di serie temporali (*Time Series Classification*, TSC) è uno dei task di machine learning più diffuso, con applicazioni critiche in vari settori come la sanità, la finanza e la cybersecurity. Negli ultimi anni sono stati presentati vari modelli di classificazione di serie temporali. Il presente lavoro di tesi si propone di effettuare un benchmarking comparativo tra alcuni dei metodi di TSC allo stato dell'arte, valutandoli su vari dataset reali e sintetici provenienti dall'archivio UCR per testarne la scalabilità e la capacità di generalizzazione. L'analisi valuta comparativamente l'efficacia predittiva (*accuracy*) e l'efficienza computazionale dei modelli, misurata in termini di tempi di addestramento (*fit*) e inferenza (*predict*). Dai risultati emerge come ROCKET si confermi l'algoritmo più efficiente, con tempi di esecuzione nell'ordine dei 10^2 secondi e prestazioni estremamente competitive. Parallelamente, HIVE-COTE 2.0 si afferma come *gold standard* per l'accuratezza, a fronte però di un costo computazionale superiore di tre ordini di grandezza ($> 10^5$ secondi) e di requisiti di memoria RAM non trascurabili. Tali evidenze sottolineano come la scelta del classificatore debba essere guidata dalle caratteristiche del dominio del problema, bilanciando accuratamente le prestazioni predittive con i vincoli computazionali esistenti.

Dichiaro di essere responsabile del contenuto dell'elaborato che presento al fine del conseguimento del titolo, di non avere plagiato in tutto o in parte il lavoro prodotto da altri e di aver citato le fonti originali in modo congruente alle normative vigenti in materia di plagio e di diritto d'autore. Sono inoltre consapevole che nel caso la mia dichiarazione risultasse mendace, potrei incorrere nelle sanzioni previste dalla legge e la mia ammissione alla prova finale potrebbe essere negata.

Capitolo 1

Introduzione

Durante gli ultimi due decenni, la classificazione di serie temporali (TSC) è diventata uno dei problemi critici nel data mining e nel machine learning [1]. Una serie temporale è una collezione di dati o valori ordinati nel tempo che, di solito, vengono registrati da dispositivi come i sensori, il cui utilizzo è aumentato vertiginosamente in moltissimi settori [2, 3]: dalla medicina, con gli elettrodi per l'ECG, ad ambiti come il monitoraggio dei terremoti o le rilevazioni meteorologiche, fino all'automazione industriale.

La crescente dipendenza dall'uso di questi sensori ha portato all'aumento della disponibilità di dati temporali e al bisogno di sviluppare algoritmi per la TSC che siano capaci di lavorare su larga scala; proprio per questo, a partire dal 2015, sono stati proposti centinaia di nuovi algoritmi per la TSC [1]. La TSC viene definita come l'area del machine learning dedicata alla categorizzazione (o etichettatura) di serie temporali [4]. Possiamo dire che qualsiasi problema di classificazione che utilizzi dati registrati tenendo conto di una qualche nozione di ordinamento possa essere interpretato come un problema di TSC [1].

Il problema della classificazione può sembrare facile ed intuitivo per un essere umano, che possiede un'abilità innata nel riconoscimento dei pattern visivi; infatti, fin da piccoli impariamo a categorizzare il mondo per analogia. Tuttavia, questo compito diventa molto complesso per un computer. Non è banale estrarre un modello statistico dalle serie temporali, poiché queste possono essere non stazionarie e mostrare proprietà statistiche variabili nel tempo. Gli algoritmi di data mining, d'altra parte, perdono efficacia a causa dell'alta dimensionalità delle serie temporali e del rumore [2].

Cercando di imitare l'approccio umano, si è giunti allo sviluppo di classificatori che comparano serie intere usando una misura di distanza o similitudine. Per lungo tempo, il classificatore di riferimento per la TSC è stato l'algoritmo nearest neighbor combinato con la metrica di similarità Dynamic Time Warping (NN-DTW), che tutt'ora viene utilizzato come *baseline* per comparare le prestazioni [5].

Tuttavia, l'esplosione del volume dei dati e la necessità di catturare pattern complessi hanno portato alla nascita di nuove famiglie di modelli. Tra questi spiccano gli approcci basati su trasformazioni casuali attraverso i random convolutional kernels, come ROCKET, che puntano a massimizzare la velocità computazionale senza sacrificare la precisione. Questa caratteristica li rende ideali per scenari con limitate risorse computazionali o dataset di dimensioni massive [6].

All'estremo opposto, per massimizzare l'accuratezza predittiva, sono stati sviluppati i cosiddetti meta-ensemble, tra cui spicca HIVE-COTE 2.0. Quest'ultimo

rappresenta lo stato dell'arte per precisione, poiché combina diverse famiglie di classificatori (basati su shapelet, dizionari, intervalli e frequenze) in un'unica architettura integrata. Sebbene estremamente potente, la sua natura di ensemble lo rende tuttavia molto oneroso in termini di tempi di calcolo [7].

Prendendo ispirazione dai successi ottenuti dalle reti neurali per la Computer Vision, sono state adattate con successo alcune architetture alla TSC. In particolare, InceptionTime ha dimostrato di poter raggiungere un'accuratezza comparabile ai migliori approcci di machine learning classici (come HIVE-COTE1), offrendo però il vantaggio dell'alta scalabilità tipica del deep learning, specialmente quando si dispone di accelerazione hardware tramite GPU [4].

In questo scenario, che offre numerose soluzioni, diventa cruciale la scelta del classificatore più adatto in base ai requisiti di accuratezza, ai vincoli computazionali e alle caratteristiche intrinseche dei dati stessi.

Il presente lavoro di tesi si pone l'obiettivo di analizzare criticamente queste diverse metodologie attraverso un benchmark comparativo. Utilizzando dataset reali e sintetici provenienti dall'archivio UCR (University of California, Riverside) [8], la sperimentazione valuterà le performance dei modelli secondo varie metriche fondamentali:

- Efficacia predittiva (*Accuracy*): testando la capacità di generalizzazione dei modelli sia su dataset estesi che in scenari con scarsità di campioni, (rischio di *overfitting*).
- Efficienza computazionale: misurando i tempi di addestramento (*fit*) e di inferenza (*predict*), con particolare attenzione alla scalabilità degli algoritmi.

I risultati ottenuti delineano un quadro chiaro dei punti di forza e delle limitazioni di ciascun approccio. HIVE-COTE 2.0 si è confermato il metodo più accurato, pur registrando la complessità computazionale massima e presentando limiti strutturali su dataset critici come Fungi ed ElectricDevices. Di contro, ROCKET si è consolidato come l'algoritmo più veloce, posizionandosi per accuratezza subito dopo HC2.

Complessivamente, l'analisi evidenzia come non esista un singolo algoritmo capace di dominare la totalità dei casi d'uso; ogni metodo eccelle infatti su specifiche tipologie di problemi. HC2 si è dimostrato competitivo in modo quasi universale, a fronte di una richiesta di risorse esorbitante. In conclusione, emerge chiaramente come la selezione dell'algoritmo di classificazione debba essere guidata da un'analisi preliminare del problema, dai requisiti di accuratezza e dalla potenza di calcolo disponibile.

Capitolo 2

Contesto Teorico e Stato dell'arte

In questa sezione vengono introdotte le definizioni preliminari necessarie alla comprensione del problema, seguite da una rassegna dei principali algoritmi che definiscono l'attuale stato dell'arte per la TSC. Si tratta di modelli che, sulla base della letteratura scientifica più recente, hanno dimostrato prestazioni di eccellenza e che, per tale ragione, saranno l'oggetto della fase sperimentale del presente lavoro.

2.1 Contesto Teorico

Nella presente tesi l'analisi si è concentrata sulle serie temporali univariate.

Definizione 1. Una serie temporale univariata $X = [x_1, x_2, \dots, x_T]$ è un insieme ordinato di T valori reali, dove T rappresenta la lunghezza della serie. A differenza delle serie multivariate, in cui vengono monitorate simultaneamente più grandezze per ogni istante temporale, una serie univariata registra le variazioni di una singola variabile nel tempo.

Definizione 2. Un dataset $D = \{(X_1, Y_1), (X_2, Y_2), \dots, (X_N, Y_N)\}$ è una collezione di N coppie (X_i, Y_i) , dove ogni X_i rappresenta una serie temporale (univariata o multivariata) e Y_i la relativa etichetta di classe.

L'obiettivo della TSC consiste nell'addestrare un classificatore su un dataset D in modo che sia capace di prevedere l'etichetta di classe per una nuova serie non osservata. A seconda dell'architettura scelta, tale output può assumere diverse forme:

- **Assegnazione diretta:** il modello assegna la serie a una specifica classe (approccio tipico dei metodi basati sulla distanza o delle trasformazioni lineari come *ROCKET*).
- **Distribuzione di probabilità:** il modello restituisce una distribuzione di confidenza probabilistica sulle classi possibili (caratteristico delle reti neurali profonde come *InceptionTime* o *ResNet*).

2.2 Classificatori di Stato dell'Arte

Le tecniche utilizzate per la TSC possono essere categorizzate in due classi: metodi basati sull'intera serie (*whole series-based*) e metodi basati sulle *feature* (caratteristiche). Per i metodi basati sull'intera serie si utilizzano delle misure di similarità che

applicano un confronto punto a punto dell'intera TS. Queste includono la Distanza Euclidea (ED) 1-NN o il Dynamic Time Warping (DTW) 1-NN, che è comunemente usato come *baseline* nei confronti. Al contrario, i classificatori basati sulle feature si affidano al confronto di caratteristiche generate da sottostrutture di TS. Nei prossimi paragrafi verranno spiegate le famiglie e gli algoritmi basati su feature più importanti che saranno l'oggetto del benchmark [3].

2.2.1 Metodi basati su Dizionari (Dictionary-based)

Questo tipo di classificatori distingue le serie temporali in base alla frequenza di ripetizione di alcune sottoserie. In particolare, abbiamo analizzato le prestazioni di BOSS e WEASEL.

2.2.1.1 BOSS ensemble (Bag-of-SFA-Symbols)

Il classificatore BOSS [2] è un metodo basato su dizionario che trasforma le serie temporali in istogrammi di parole simboliche. Il suo funzionamento può essere suddiviso nelle seguenti fasi operative:

Windowing e Normalizzazione. Per prima cosa, BOSS estrae delle sottosequenze (segmenti) attraverso una finestra che scorre sulla serie temporale (*Windowing*). Ogni segmento estratto viene normalizzato per avere deviazione standard pari a 1 al fine di ottenere l'invarianza all'ampiezza: nello specifico, si sottrae la media per centrare il segmento sullo zero (allineamento verticale) e si divide per la deviazione standard per uniformare l'altezza dei picchi.

Symbolic Fourier Approximation (SFA). Il passo seguente è la SFA, applicata ai segmenti estratti con l'obiettivo di trasformare la serie originale in un set non ordinato di parole SFA. Questa procedura si divide in due passaggi:

- **Analisi in frequenza:** si applica la Trasformata di Fourier Discreta (DFT), che agisce come un filtro passa-basso rimuovendo il rumore dal segnale. La lunghezza della parola SFA determina il numero di coefficienti di Fourier da utilizzare e, di conseguenza, la larghezza di banda del filtro.
- **Quantizzazione:** il secondo passo è la conversione dei dati in stringhe di caratteri tramite la *Multiple Coefficient Binning* (MCB). Questo approccio consente l'applicazione di algoritmi di *string matching*. La dimensione dell'alfabeto definisce il grado di quantizzazione, che riduce ulteriormente il rumore pur potendo comportare una parziale perdita di informazioni [2].

Riduzione della numerosità e Istogrammi. In un segmento stabile di un segnale, le parole SFA di due finestre scorrevoli adiacenti hanno un'alta probabilità di essere identiche. Per evitare che tali sezioni assumano un peso eccessivo, viene applicata la "riduzione della numerosità", registrando solo la prima occorrenza di una parola e ignorando i duplicati consecutivi. A partire dalle parole ottenute, viene costruito un **istogramma BOSS** che conta le occorrenze totali di ciascun pattern, ignorandone l'ordine temporale per garantire l'invarianza di fase. La somiglianza tra due

serie viene calcolata tramite la *Distanza BOSS*, una metrica che confronta gli istogrammi basandosi sulle parole presenti nel campione da classificare, massimizzando l'accuratezza anche in presenza di rumore.

Classificazione Ensemble. Il modello si basa su un classificatore *1-Nearest Neighbor* (1-NN). BOSS opera come un **ensemble**: in ambito *Machine Learning*, un ensemble prevede l'addestramento di più modelli per risolvere lo stesso problema, combinando i risultati delle singole predizioni attraverso media o voto di maggioranza al fine di aumentare l'accuratezza e l'affidabilità finale. Nello specifico, BOSS addestra diverse configurazioni dello stesso modello eseguendo una ricerca sistematica (*grid-search*) per individuare le lunghezze di finestra più efficaci. Invece di selezionare un unico parametro, vengono trattenuti tutti i classificatori che garantiscono un'accuratezza elevata (entro una soglia definita); i risultati di questi modelli vengono poi combinati attraverso una votazione a maggioranza. L'onere computazionale principale di BOSS risiede nella fase di addestramento, che presenta una complessità di $\mathcal{O}(N^2T^2)$ [2].

2.2.1.2 WEASEL (Word ExtrAction for time SEries cLassification)

L'algoritmo WEASEL [3] si basa concettualmente sul modello BOSS, ma introduce diversi accorgimenti per ottimizzare la selezione delle finestre e delle parole discriminanti, migliorando significativamente sia l'accuratezza che l'efficienza computazionale.

Trasformata di Fourier Troncata e Anova F-test. La prima differenza risiede nell'utilizzo della Trasformata di Fourier Troncata (TFT), che permette di calcolare esclusivamente i primi coefficienti di bassa frequenza e di aggiornarli in modo incrementale durante lo scorrimento della finestra (*sliding window*). Successivamente, tramite il test statistico *ANOVA F-test*, vengono selezionati solo i coefficienti con il maggior potere discriminante tra le classi, riducendo lo spazio delle feature di una percentuale compresa tra il 30 e il 70%.

Discretizzazione Supervisionata. Nella fase di discretizzazione, WEASEL applica una quantizzazione supervisionata sui coefficienti di Fourier precedentemente selezionati. L'algoritmo ricerca i punti di divisione (*split points*) che massimizzano l'*Information Gain*, definendo intervalli (*bin*) che garantiscano la massima purezza possibile.

Bigrammi e Strategia Multi-scala. Per mitigare la perdita dell'ordine delle sottostrutture tipica degli approcci *Bag-of-Words*, WEASEL considera i bigrammi di parole come feature, codificando così l'ordine locale dei pattern. Inoltre, adotta una strategia multi-scala estraendo simultaneamente parole da finestre di diverse lunghezze. I risultati vengono uniti in un unico vettore di feature, consentendo al modello di riconoscere pattern che si manifestano su scale temporali differenti.

Selezione delle Feature e Classificazione. L'inclusione di bigrammi e l'uso di finestre variabili causano un'espansione esponenziale dello spazio delle feature. Per mantenere l'efficienza, WEASEL implementa una selezione delle feature aggressiva

tramite il test Chi-quadro (χ^2), diminuendo drasticamente il numero di variabili senza impattare negativamente sull'accuratezza. Infine, il vettore di feature altamente discriminante costruito, viene fornito in input ad un classificatore basato sulla **regressione logistica**, che ha una complessità computazionale lineare [3].

2.2.2 Metodi basati su Convoluzioni Casuali

2.2.2.1 ROCKET (RandOm Convolutional Kernel Transform)

L'introduzione di ROCKET [6] ha segnato un punto di svolta nella classificazione di serie temporali, dimostrando che è possibile raggiungere livelli di *accuracy* allo stato dell'arte con una frazione del tempo utilizzato dagli algoritmi più recenti, grazie all'impiego di *random convolutional kernels*.

Il Concetto di Kernel Convoluzionale. Un kernel convoluzionale è un vettore di pesi che esegue un prodotto scalare a scorrimento (*sliding dot product*) lungo la serie temporale, con lo scopo di trasformare quest'ultima in una mappa di caratteristiche (*feature map*). ROCKET utilizza un numero molto elevato di kernel (solitamente 10.000) poiché applica un singolo layer di convoluzioni, caratterizzato da un costo computazionale estremamente ridotto.

Parametri Casuali. A differenza delle classiche reti neurali convoluzionali, ROCKET utilizza kernel del tutto casuali. Ogni parametro del kernel — inclusi lunghezza, pesi, *bias*, dilatazione e *padding* — viene generato in modo stocastico, eliminando la necessità di una fase di addestramento per l'estrazione delle feature [6].

Estrazione delle Feature: PPV e Max Value. Da ogni kernel applicato alla serie temporale, ROCKET estrae due valori fondamentali:

- **PPV (Proportion of Positive Values):** Questa metrica indica la percentuale della serie che combacia con un pattern, permettendo al modello di capire quanto sia prevalente un certo andamento.
- **Max Value:** Rappresenta l'intensità massima del pattern individuato lungo la serie.

Classificazione e Complessità Computazionale. Queste due metriche vengono utilizzate per addestrare un classificatore lineare (solitamente una *Ridge Regression*), capace di gestire efficacemente un numero elevato di feature estratte ma singolarmente "poco informative". La fase di trasformazione delle serie ha una complessità lineare rispetto alla lunghezza della serie T , al numero di campioni N e al numero di kernel k , definita come $\mathcal{O}(k \cdot N \cdot T)$. Il carico computazionale principale risiede nell'addestramento del classificatore, operazione quadratica che dipende dal rapporto tra il numero di serie N e il numero di feature estratte f :

- Se $N > f$: $\mathcal{O}(N \cdot f^2)$
- Se $f > N$: $\mathcal{O}(N^2 \cdot f)$

2.2.3 Metodi basati su Intervalli (Interval-based)

2.2.3.1 Random Interval Classifier (RIC)

Il *Random Interval Classifier* (RIC) opera estraendo dalle serie temporali originali, intervalli di lunghezza, posizione e dimensione casuali.

Estrazione delle Feature: Catch22. Ciascun segmento estratto viene successivamente trasformato tramite il trasformatore **Catch22** (*CAnnonical Time-series CHaracteristics*) [9]. Questo modulo consiste in un set di 22 caratteristiche statistiche selezionate per la loro efficacia predittiva a partire dal vasto pool *hctsa* (che comprende oltre 4700 caratteristiche). Tale processo permette di ottenere un vettore di feature di dimensione moderata ma altamente informativo, capace di sintetizzare efficacemente le proprietà dinamiche locali della serie.

Classificazione: Rotation Forest. Le caratteristiche estratte alimentano una *Rotation Forest* [10], un classificatore *ensemble* composto da diversi alberi di decisione. A differenza delle foreste casuali tradizionali, questo modello aumenta la diversità tra i classificatori di base attraverso la manipolazione dello spazio delle feature:

- Il set di attributi viene suddiviso casualmente in K sottoinsiemi disgiunti.
- A ciascun sottoinsieme viene applicata l'Analisi delle Componenti Principali (PCA).
- Per preservare l'intera variabilità dei dati, vengono mantenute tutte le componenti principali; queste vengono utilizzate per ruotare gli assi del dataset originale, costruendo così le nuove feature di input per ogni albero.

2.2.4 HIVE-COTE 2.0

HIVE-COTE (*Hierarchical Vote Collective of Transformation-based Ensembles*) 2.0 [7] è un *meta-ensemble* ("ensemble di ensemble") eterogeneo, composto da classificatori specializzati su diversi domini di rappresentazione quali *shapelet*, dizionari, intervalli e convoluzioni.

HIVE-COTE 2.0 rappresenta un significativo aggiornamento rispetto alla versione 1.0, avendo sostituito tre dei quattro classificatori originali con l'introduzione di *Temporal Dictionary Ensemble* (TDE), *Diverse Representation Canonical Interval Forest* (DrCIF) e *Arsenal* (un ensemble basato su ROCKET). Inoltre, la versione 2.0 apporta miglioramenti ai restanti componenti, in particolare allo *Shapelet Transform Classifier* (STC) e all'unità di controllo CAWPE (*Cross-validation Accuracy Weighted Probabilistic Ensemble*) [7].

Quest'ultima unità ha il compito di stimare l'affidabilità di ciascuna delle quattro componenti di HC2 durante la fase di addestramento, assegnando pesi probabilistici proporzionali alle loro prestazioni; tali pesi verranno poi utilizzati nella fase di predizione finale [7].

Per quanto riguarda la costruzione dei modelli, HC2 adotta un approccio ibrido. Per tre dei suoi componenti (STC, DrCIF e Arsenal), vengono addestrate due versioni distinte:

Modello per la stima (Bagging/OOB). Un ensemble addestrato tramite *bagging* che permette una valutazione rapida dell'accuratezza attraverso i dati *Out-of-Bag* (OOB). Questa stima è fondamentale per consentire a CAWPE di calcolare i pesi di ponderazione.

Modello per la predizione. Un secondo modello addestrato sull'intero dataset di training al fine di massimizzare la capacità predittiva finale.

A differenza degli altri, il modulo TDE genera un unico modello poiché, essendo già addestrato per sua natura su sottoinsiemi di dati, fornisce autonomamente una stima OOB valida per la pesatura [7].

2.2.4.1 Temporal Dictionary Ensemble (TDE)

TDE [7] è un ensemble di classificatori *1-Nearest Neighbor* (1-NN) che condivide i principi fondamentali con l'algoritmo BOSS. Esso applica la trasformazione SFA alle finestre scorrevoli per discretizzarle in parole e generare un istogramma delle occorrenze, calcolando la somiglianza tra serie attraverso la metrica dell'intersezione tra istogrammi. In aggiunta, TDE cattura la frequenza dei bigrammi tra finestre non sovrapposte e integra informazioni spaziali mediante l'utilizzo di piramidi spaziali. Gli istogrammi delle frequenze vengono concatenati e pesati in base al relativo livello di dettaglio analizzato [7]. Per la discretizzazione delle parole SFA, l'algoritmo può optare sia per il metodo MCB (*Multiple Coefficient Binning*), tipico di BOSS, sia per l'*Information Gain* utilizzato in WEASEL. TDE seleziona i migliori classificatori per costituire il proprio ensemble stimandone le prestazioni tramite *leave-one-out cross-validation* (LOOCV) e garantisce la diversità del modello alterando i parametri per ogni classificatore e utilizzando al massimo il 70% del *training set* [7].

2.2.4.2 Diverse Representation Canonical Interval Forest (DrCIF)

DrCIF [7] è un ensemble progettato per estrarre sottosequenze dipendenti dalla fase al fine di individuare caratteristiche discriminanti su vari intervalli. Il suo classificatore di base è il *Time Series Tree*, un albero decisionale basato sull'*Information Gain*. Le caratteristiche fornite all'albero derivano da molteplici intervalli estratti da tre diverse rappresentazioni: la serie temporale originale, la serie delle differenze del primo ordine e i periodogrammi delle serie intere [7]. Gli intervalli vengono selezionati casualmente da ciascuna rappresentazione. DrCif utilizza un pool di 29 statistiche totali che comprendono: media, deviazione standard, pendenza, mediana, intervallo interquartile, minimo, massimo e le caratteristiche matematiche fornite dal set *catch22* [9]. Per ogni albero viene scelto un sottoinsieme casuale di queste 29 feature. Per ciascuna delle tre rappresentazioni, vengono estratti intervalli dipendenti dalla fase con posizioni e lunghezze casuali; le caratteristiche selezionate sono quindi calcolate per ogni intervallo e concatenate per costruire il dataset finale utilizzato dall'albero. La diversità dell'ensemble è garantita proprio dalla scelta stocastica dei sottoinsiemi di feature e degli intervalli estratti [7].

2.2.4.3 Arsenal: un ensemble di ROCKET

Lo sviluppo di Arsenal è stato motivato dalle difficoltà tecniche nell'integrare l'algoritmo ROCKET [6] all'interno del modulo di controllo CAWPE; quest'ultimo utiliz-

za infatti probabilità pesate che risultano difficili da configurare con il classificatore *Ridge Regression* originariamente impiegato da ROCKET. Per superare tale limite, è stato sviluppato Arsenal, un ensemble composto da molteplici classificatori ROCKET di dimensioni ridotte [7]. Nonostante Arsenal richieda tempi di costruzione superiori rispetto alla versione singola a causa della sua natura di ensemble, la sua struttura permette di ottenere le probabilità di appartenenza alle classi attraverso la votazione dei singoli componenti. Ciò lo rende un candidato ideale per l'integrazione nel meta-ensemble HC2, garantendo al contempo una maggiore robustezza e una stima più accurata dell'incertezza nelle predizioni finali [7].

2.2.4.4 Shapelet Transform Classifier (STC)

Il modulo STC effettua una ricerca esaustiva e stocastica delle migliori *shapelet* entro un limite di tempo prestabilito [7]. Le *shapelet* sono sottoserie indipendenti dalla fase che vengono selezionate attraverso l'impiego dell'Information Gain (IG), il quale permette di filtrare le sottosequenze che distinguono con maggiore efficacia una classe dall'altra, scartando quelle con un valore di IG ridotto. Una volta completata la fase di ricerca, viene eseguita la trasformazione del dataset originale: per ogni serie temporale viene calcolata la distanza euclidea quadratica minima rispetto a ciascuna delle *shapelet* individuate [11]. Il risultato di tale processo è un nuovo set di dati che viene utilizzato come input per la costruzione di una *Rotation Forest* [10], completando così l'architettura del classificatore [7].

2.2.5 Deep Learning per la TSC

A partire dal 2015, si è assistito a una progressiva applicazione delle architetture di *Deep Learning* — originariamente concepite per la *Computer Vision* — in ambito di classificazione di serie temporali. Nel presente lavoro, vengono analizzate le prestazioni di due tra i modelli più innovativi in questo campo: *ResNet* e *InceptionTime*.

2.2.5.1 ResNet

La *ResNet* (*Residual Network*) [12] è una rete neurale profonda che introduce delle **connessioni scorciatoia** (*shortcut connections*) tra i blocchi residui che la compongono. Tali connessioni permettono al gradiente di fluire direttamente attraverso gli strati inferiori della rete durante la fase di addestramento [12]. La struttura della ResNet è basata su una gerarchia di blocchi residui, a loro volta composti da blocchi convoluzionali. Seguendo l'architettura della *FCN* (*Fully Convolutional Network*) [12], ogni blocco convoluzionale è costituito da uno strato di convoluzione, seguito da uno strato di *batch normalization* e da uno strato di attivazione *ReLU*. L'operazione di convoluzione viene eseguita da tre kernel monodimensionali scorrevoli (1-D kernels) con dimensioni {8, 5, 3} e passo unitario (*without striding*). Nello specifico, ogni blocco residuo è formato da tre strati convoluzionali in cascata, dove l'output di ciascuno funge da input per il successivo. All'output del terzo strato convoluzionale viene sommato l'input originale del blocco tramite la connessione residua; il risultato di tale somma passa attraverso un'ultima funzione di attivazione *ReLU*, che produce l'input per il blocco residuo successivo [12]. La ResNet utilizza filtri di dimensioni {64, 128} come segue: il primo blocco residuo impiega 64 filtri per ognuno dei suoi

strati convoluzionali, mentre i restanti due blocchi ne utilizzano 128. La struttura complessiva della rete prevede dunque una sequenza di tre blocchi residui, seguiti da uno strato di *Global Average Pooling* (GAP) e da uno strato *Softmax* finale per la classificazione [12].

2.2.5.2 InceptionTime

InceptionTime [4] è un modello *ensemble* composto da cinque reti *Inception* indipendenti, inizializzate casualmente. Il componente principale dell'architettura è il modulo Inception, che sostituisce i tradizionali blocchi convoluzionali utilizzati in modelli come ResNet [4]. Il classificatore è strutturato in due blocchi residui, ciascuno dei quali è composto da tre moduli Inception. Come nella ResNet, l'input di ogni blocco residuo viene sommato all'input del blocco successivo tramite una connessione scorciatoia (*shortcut connection*), così da mitigare il problema della scomparsa del gradiente. Al termine di ognuno di questi blocchi, uno strato di *Global Average Pooling* (GAP) calcola la media dei valori della serie temporale multivariata in uscita lungo l'intera dimensione temporale. Infine, uno strato *softmax*, con un numero di neuroni pari alle classi presenti nel dataset, fornisce il risultato della classificazione [4].

Il Modulo Inception. Il modulo Inception è progettato per estrarre caratteristiche grazie all'impiego simultaneo di filtri di varie lunghezze [4]. La sua struttura si articola in tre componenti principali:

- **Strato Bottleneck:** Il primo componente è uno strato di bottleneck che riduce la dimensionalità della serie temporale multivariata (MTS) in input. Attraverso l'applicazione di filtri di lunghezza 1 e passo unitario, questo strato trasforma la serie originale riducendone il numero di canali. Tale riduzione permette alla rete di utilizzare filtri molto più lunghi rispetto a quelli della ResNet senza aumentare la complessità computazionale [4].
- **Filtri Convoluzionali Paralleli:** L'output dello strato bottleneck viene elaborato simultaneamente da tre set di filtri scorrevoli con kernel di lunghezze diverse, nello specifico $\{10, 20, 40\}$. Questa varietà permette al modello di catturare pattern con diverse estensioni temporali.
- **MaxPooling:** Parallelamente alle convoluzioni, viene eseguita un'operazione di *MaxPooling* per rendere il modello invariante a piccole perturbazioni. Anche l'output di questa operazione viene fatto passare attraverso uno strato di bottleneck per uniformarne la dimensionalità. Gli output di ciascun set di filtri e del *MaxPooling* vengono infine concatenati per formare la serie temporale multivariata in uscita dal modulo. Nello specifico, ogni modulo utilizza 3 set da 32 filtri per le convoluzioni (uno per ogni lunghezza) e un set da 32 per il *MaxPooling*, per un totale di 128 filtri per strato, che definiscono la dimensionalità della serie in output [4]. Poiché una singola rete *Inception* tende a mostrare un'elevata deviazione standard nell'accuratezza a causa dell'inizializzazione casuale dei pesi, si è optato per l'uso in ensemble. InceptionTime combina le predizioni di cinque reti diverse, stabilizzando i risultati e migliorando le prestazioni rispetto al singolo modello [4].

2.3 Esperimenti e risultati

2.3.1 Configurazione sperimentale

Gli esperimenti sono stati eseguiti in un ambiente virtuale di Python 3.10, versione scelta per la stabilità. Tutti i classificatori e le relative metriche sono stati presi dalle implementazioni ufficiali della libreria python sktime. Per quanto riguarda l'hardware che il centro internazionale di HPC ha messo a disposizione, gli esperimenti di Machine-Learning sono stati eseguiti su una macchina virtuale equipaggiata con 8 CPU virtuali basate su architettura Intel Xeon Cascadelake (2019), operanti senza hyper-threading, garantendo efficienza e stabilità, supportate da 31GB di RAM. Invece per gli esperimenti di Deep-Learning è stata messa a disposizione una GPU NVIDIA Tesla T4 dotata di 16GB di memoria VRAM, così da poter sfruttare l'accelerazione hardware per ottimizzare i tempi di calcolo.

Dataset	Train size	Test size	Length	Class	Type
ElectricDevices	8926	7711	96	7	Device
Crop	7200	16800	46	24	Image
FordB	3636	810	500	2	Sensor
HandOutlines	1000	370	2709	2	Image
HouseTwenty	40	119	2000	2	Device
Rock	20	50	2844	4	Spectrum
DiatomSizeReduction	16	306	345	4	Image
InsectEPGSmallTrain	17	249	601	3	EPG
Fungi	18	186	201	18	HRM
SmoothSubspace	150	150	15	15	Simulated
ItalyPowerDemand	67	1029	24	2	Sensor
Chinatown	20	343	24	2	Traffic

Tabella 2.1: Descrizione statistiche dei Datasets UCR

2.3.2 Dataset utilizzati

I dataset utilizzati (tabella 2.1) per la valutazione dei classificatori provengono dall'archivio UCR (University of California, Riverside) [8], riconosciuto come standard per la classificazione di serie temporali a livello scientifico.

Sono stati scelti 12 datasets con l'obiettivo di analizzare le prestazioni al variare del numero di serie temporali per l'addestramento e della loro lunghezza. Nello specifico, per testare scenari al limite, si sono scelti i seguenti criteri:

- **i 3 dataset con Train size maggiore:** scelti per valutare la scalabilità degli algoritmi e la loro capacità di gestire grandi quantità di dati.
- **i 3 dataset con Train size minore:** selezionati per testare la capacità di generalizzazione dei modelli, in presenza di pochi campioni, ovvero la capacità di non sfociare nell'overfitting.
- **i 3 dataset con serie di lunghezza maggiore:** inseriti per testare la capacità di distinguere segnali macroscopici che possono essere diluiti nel rumore.

- **i 3 dataset con serie di lunghezza minore:** utilizzati per analizzare la robustezza e la capacità degli algoritmi di imparare pattern di segnali brevi o con scarsa risoluzione temporale.

Procedure di Addestramento e Pre-processing. I risultati presentati per ogni algoritmo rappresentano la media di tre diversi *seed* per ogni dataset valutato. Fa eccezione HIVE-COTE 2.0, per il quale, data l'elevata complessità computazionale che comporta tempi di addestramento estremamente lunghi, si è deciso di utilizzare un singolo *seed* per rendere fattibile il confronto prestazionale. Ogni esecuzione è stata parallelizzata su 8 core per ottimizzare i tempi di calcolo. Prima della fase di classificazione, tutte le serie temporali sono state sottoposte a *z-normalizzazione* per ottenere media uguale a zero e deviazione standard pari a uno, procedura considerata una *best practice* prima della classificazione di serie temporali [5].

Parametri e Limiti Temporal. Tutti gli algoritmi sono stati eseguiti utilizzando i parametri di *default* proposti nei rispettivi lavori originali o nelle implementazioni standard della libreria *sktime*. Anche in questo caso, è stato necessario apportare modifiche per HIVE-COTE 2.0: oltre al limite temporale standard di 2 ore per il componente STC, sono stati imposti limiti di 3 ore anche per i moduli TDE e DrCIF. Per garantire un confronto equo, STC e DrCIF sono stati valutati anche singolarmente al di fuori del *meta-ensemble* HC2, mantenendo i medesimi limiti temporali ma estendendo la valutazione su tre *seed*.

Ottimizzazione per Deep Learning. Per quanto riguarda la *ResNet*, è stato necessario ridurre il *learning rate* dell'ottimizzatore Adam (*Adaptive Moment Estimation*) esclusivamente per il dataset *ElectricDevices*, passando dal valore predefinito di 0,001 a 0,0001. Tale variazione è stata introdotta per prevenire l'esplosione del gradiente (*exploding gradient*) riscontrata specificamente su questo dataset, fenomeno che portava i pesi della rete verso valori infiniti, risultando in valori **NaN** (*Not a Number*) e rendendo impossibile il calcolo delle probabilità di output.

Limitazioni e Casi Critici. Nonostante i limiti temporali imposti, HIVE-COTE 2.0 ha presentato criticità legate alla complessità spaziale e alla gestione della memoria, manifestando fenomeni di *deadlock* da saturazione di RAM sui dataset più estesi come *Crop*, *FordB* e, in particolare, *ElectricDevices*, per il quale non è stato possibile ottenere risultati. È opportuno sottolineare che l'utilizzo di una macchina dotata di maggiore memoria RAM avrebbe, permesso di gestire anche tali moli di dati. Anche il dataset *Fungi* è stato escluso dalla valutazione con HC2 a causa della sua natura peculiare: esso presenta infatti 18 classi e solo 18 serie temporali per il *training set*, una per ogni classe. L'architettura di HC2 e del controllore CAWPE prevede la costruzione di due modelli per ogni componente: uno per la predizione (addestrato sul set completo) e uno per la stima dell'accuratezza tramite *bagging* su dati *Out-of-Bag* (OOB). In presenza di un solo campione per classe, se questo viene incluso nel set di addestramento, risulta assente dai dati OOB per la validazione; viceversa, se escluso dal set di addestramento, il modello verrebbe addestrato senza alcuna informazione su quella specifica classe. Questa condizione impedisce a CAWPE di calcolare pesi probabilistici affidabili, portando all'annullamento dei pesi dei componenti e alla generazione di valori **NaN** in fase di predizione.

2.4 Analisi dei Risultati

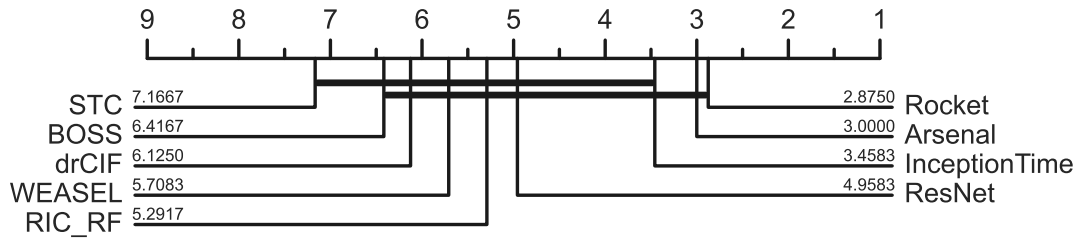


Figura 2.1: Critical Difference Diagram per il confronto dei rank medi, esclude HiveCote2.0 e analizza il resto dei metodi su tutti i dataset.

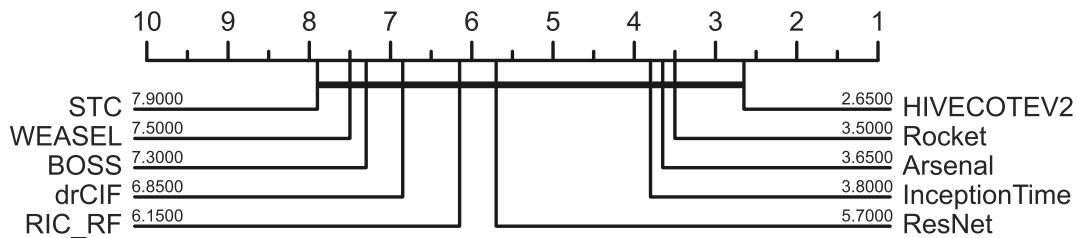


Figura 2.2: Critical Difference Diagram per il confronto dei rank medi, comprende HiveCote2.0 escludendo però i dataset Fungi e ElectricDevices.

2.4.1 Accuratezza dei modelli

La Figure 2.2 e 2.1 illustrano il *rank* medio per ciascun metodo incluso nel confronto. Nella rappresentazione, i classificatori con *rank* più basso — dove un valore inferiore indica prestazioni migliori — sono posizionati verso destra. I modelli le cui differenze di accuratezza non risultano statisticamente significative sono uniti da una linea nera continua; tale significatività è stata determinata attraverso il Wilcoxon signed-rank test con correzione di Holm (come test post-hoc rispetto al test di Friedman) [6].

Nello specifico, la Figura 2.2 confronta tutti i classificatori sui 10 dataset (su un totale di 12) per i quali sono disponibili i risultati di HIVE-COTE 2.0. Si può osservare come HC2, nonostante i vincoli temporali imposti, si confermi il classificatore con il *rank* medio più basso, sebbene il distacco rispetto a ROCKET, Arsenal e InceptionTime non sia netto. L'assenza di una differenza statisticamente significativa tra HC2 e i restanti metodi è verosimilmente imputabile al numero limitato di dataset testati, che riduce la potenza del test statistico.

A conferma di questa ipotesi, la Figura 2.1 analizza l'intero insieme dei 12 dataset (inclusando Fungi e ElectricDevices, esclusi dal grafico precedente) ma omettendo HIVE-COTE 2.0. In questo caso, l'aumento del campione permette di delineare una differenza statisticamente rilevante tra l'accuratezza di STC e quella di ROCKET. Si può quindi dedurre che, espandendo l'analisi a un numero maggiore di dataset, si otterrebbe un distacco statisticamente significativo tra HIVE-COTE 2.0 e i classificatori con *rank* uguale o inferiore a ResNet; rimane incerto se tale distacco

diverrebbe significativo anche nei confronti di ROCKET, Arsenal e InceptionTime, data la loro vicinanza con il meta-ensemble nelle prestazioni medie.

Dataset	Arsenal	BOSS	HiveCote2	InceptionTime	RIC	RF	ResNet	Rocket	STC	WEASEL	DrCIF
Chinatown	.9825 $\pm$.000	.9067 $\pm$.000	.9825 -	.9835 $\pm$.002	.9776 $\pm$.003	.9825 $\pm$.003	.9815 $\pm$.002	.9757 $\pm$.004	.9563 $\pm$.000	.9776 $\pm$.003	
Crop	.7378 $\pm$.000	.6645 $\pm$.000	.7459 -	.7487 $\pm$.004	.7444 $\pm$.001	.7129 $\pm$.005	.7561 $\pm$.002	.6644 $\pm$.002	.6983 $\pm$.000	.6902 $\pm$.001	
DiatomSizeReduction	.9739 $\pm$.000	.9281 $\pm$.000	.9510 -	.9237 $\pm$.026	.9205 $\pm$.012	.9684 $\pm$.007	.9739 $\pm$.006	.9281 $\pm$.014	.9150 $\pm$.000	.8704 $\pm$.022	
ElectricDevices	.7280 $\pm$.002	.6732 $\pm$.002	-	.7033 $\pm$.001	.7275 $\pm$.002	.7382 $\pm$.008	.7305 $\pm$.002	.6929 $\pm$.010	.7583 $\pm$.000	.7089 $\pm$.009	
FordB	.8111 $\pm$.005	.8160 $\pm$.002	.8198 -	.8568 $\pm$.005	.7617 $\pm$.009	.8029 $\pm$.008	.8066 $\pm$.007	.7959 $\pm$.009	.8025 $\pm$.000	.7745 $\pm$.007	
Fungi	1.000 $\pm$.000	.9946 $\pm$.000	-	1.000 $\pm$.000	.9731 $\pm$.019	.8136 $\pm$.249	1.000 $\pm$.000	.7670 $\pm$.026	1.000 $\pm$.000	.8100 $\pm$.059	
HandOutlines	.9396 $\pm$.003	.9189 $\pm$.000	.9405 -	.9495 $\pm$.006	.9270 $\pm$.003	.7928 $\pm$.080	.9432 $\pm$.003	.9099 $\pm$.007	.8649 $\pm$.000	.9108 $\pm$.005	
HouseTwenty	.9664 $\pm$.000	.9748 $\pm$.000	.9748 -	.9720 $\pm$.005	.9748 $\pm$.008	.9776 $\pm$.005	.9636 $\pm$.005	.9692 $\pm$.018	.9076 $\pm$.000	.9832 $\pm$.008	
InsectEPGSmallTrain	.9813 $\pm$.002	.9277 $\pm$.000	.9719 -	.9424 $\pm$.002	.8407 $\pm$.030	.7992 $\pm$.057	.9813 $\pm$.002	.8715 $\pm$.053	.9398 $\pm$.000	.8956 $\pm$.004	
ItalyPowerDemand	.9699 $\pm$.000	.9310 $\pm$.000	.9699 -	.9653 $\pm$.002	.9517 $\pm$.001	.9602 $\pm$.008	.9692 $\pm$.001	.9498 $\pm$.003	.9631 $\pm$.000	.9559 $\pm$.004	
Rock	.9000 $\pm$.000	.7600 $\pm$.000	.9600 -	.7800 $\pm$.040	.9267 $\pm$.011	.4467 $\pm$.046	.9000 $\pm$.000	.8133 $\pm$.042	.8600 $\pm$.000	.8467 $\pm$.011	
SmoothSubspace	.9778 $\pm$.004	.5956 $\pm$.010	.9800 -	.9778 $\pm$.008	.9600 $\pm$.007	.9844 $\pm$.010	.9778 $\pm$.004	.9000 $\pm$.018	.7733 $\pm$.000	.9511 $\pm$.014	

Tabella 2.2: Analisi comparativa delle prestazioni (Media $\pm$ Std Dev) dei classificatori TSC sui dataset selezionati. Le migliori accuratèzze sono evidenziate in grassetto. Per HC2 viene riportata l'accuratèzza del singolo seed sperimentato (Std Dev non applicabile)

La Tabella 2.2 riporta l'accuratèzza media e la relativa deviazione standard, calcolata su tre *seed* distinti per ogni coppia dataset-algoritmo. I valori in grassetto evidenziano le performance migliori ottenute per ciascun dataset.

Dall'analisi dei dati emerge come nessun metodo prevalga sistematicamente su tutti gli altri, confermando che ogni algoritmo eccelle su specifiche tipologie di problemi. Sebbene il diagramma delle differenze critiche (Figura 2.2) indichi HC2 come l'algoritmo con il miglior *rank* medio, la Tabella 2.2 suggerisce una prospettiva complementare: algoritmi come Arsenal, InceptionTime e ROCKET presentano infatti il maggior numero di "vittorie" singole (accuratèzza massima assoluta).

Questa apparente contraddizione indica che, mentre questi tre modelli risultano estremamente efficaci su problemi specifici, HC2 garantisce una maggiore stabilitè prestazionale, posizionandosi costantemente tra i migliori metodi in quasi tutti i dataset. In conclusione, l'analisi evidenzia l'importanza di selezionare l'algoritmo in base allo specifico caso d'uso; tuttavia, qualora si disponga di risorse computazionali sufficienti, HC2 si conferma la scelta piÙ affidabile "*a scatola chiusa*", garantendo ottime prestazioni medie senza necessitè di analisi preliminari.

Analizzando le deviazioni standard, si osserva inoltre come i classificatori basati su *Deep Learning* — in particolare ResNet — e alcuni componenti dell'ensemble HC2, come STC e DrCIF, presentino un'accentuata instabilitè su specifici dataset (quali *Fungi*, *Rock*, *InsectEPGSmall* e *HandOutlines*).

Queste criticità si manifestano prevalentemente in presenza di dataset caratterizzati da un numero esiguo di serie temporali nel set di addestramento e/o da una notevole lunghezza delle serie stesse. Tale evidenza suggerisce che la scarsità di dati possa destabilizzare i risultati; nel caso delle reti neurali, ciò è riconducibile alla forte influenza dell'inizializzazione stocastica dei pesi sulla convergenza del modello.

Per quanto riguarda invece STC e DrCIF, l'instabilitè sembra derivare dai limiti temporali imposti alla fase di addestramento: tali vincoli possono impedire la generazione di un numero sufficiente di modelli per garantire la robustezza dell'ensemble, o limitare eccessivamente la ricerca dei parametri ottimali, compromettendo la riproducibilitè delle performance tra i diversi *seed*.

2.4.2 Efficienza computazionale

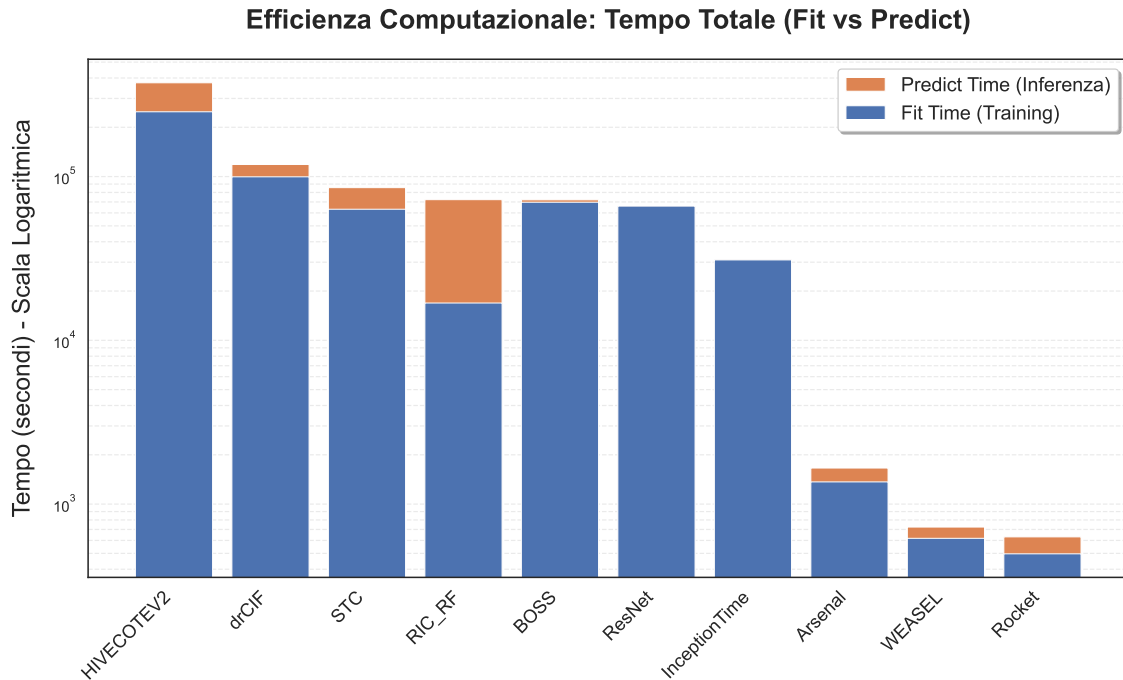


Figura 2.3: Stacked Barplot per il confronto dei tempi di esecuzione totali dei metodi in scala logaritmica.

La Figura 2.3 mostra i tempi totali di esecuzione di ogni algoritmo in scala logaritmica, distinguendo tra tempo di addestramento (in blu) e tempo di predizione (in arancione). HIVE-COTE 2.0 si dimostra l'algoritmo con la maggiore complessità computazionale, con tempi di esecuzione superiori di tre ordini di grandezza rispetto ai metodi più rapidi. Tale onere è giustificato dalla sua architettura di *meta-ensemble*, che ne garantisce però l'elevata accuratezza. Seguono gli altri *ensemble* e i modelli di Deep Learning come DrCIF, STC, RIC, BOSS, ResNet e InceptionTime; è interessante notare come ResNet risulti più lenta di InceptionTime, nonostante quest'ultimo sia un ensemble. Questa differenza è riconducibile a due fattori principali: in primo luogo, il modulo Inception utilizza vari strati di bottleneck che riducono sensibilmente la dimensionalità delle feature estratte; in secondo luogo, la ResNet presenta una struttura più profonda, basata su 3 moduli residui, contro i 2 moduli Inception previsti per ogni classificatore all'interno dell'ensemble.

I metodi più efficienti risultano essere ROCKET, seguito da WEASEL e Arsenal. In generale, i tempi di esecuzione di tutti gli algoritmi sono dominati dalla fase di addestramento. Tuttavia, per il RIC con Rotation Forest, si osserva che la fase di predizione prende una parte significativa del tempo totale. Tale comportamento è riconducibile alla pipeline dell'algoritmo: in fase di inferenza, è necessario riestrarre gli intervalli e calcolare nuovamente le statistiche di Catch22 per ogni serie temporale. Il vettore di caratteristiche risultante deve poi essere proiettato negli spazi definiti dalle rotazioni PCA apprese durante l'addestramento, operazione indispensabile affinché i dati risultino coerenti con la struttura degli alberi dell'ensemble. Al contrario per BOSS, ResNet e InceptionTime, l'inferenza risulta pressoché istantanea. Per i modelli di Deep Learning, ciò è dovuto al fatto che l'addestramento

richiede un processo iterativo di ottimizzazione dei pesi tramite *backpropagation*, mentre la predizione consiste in un singolo passaggio attraverso gli strati della rete. Per BOSS, l'onere risiede nella ricerca intensiva dei parametri ottimali (*grid search*) in fase di training, operazione molto più costosa rispetto al calcolo della *BOSS-distance* effettuato in fase di predizione solo sui migliori modelli selezionati.

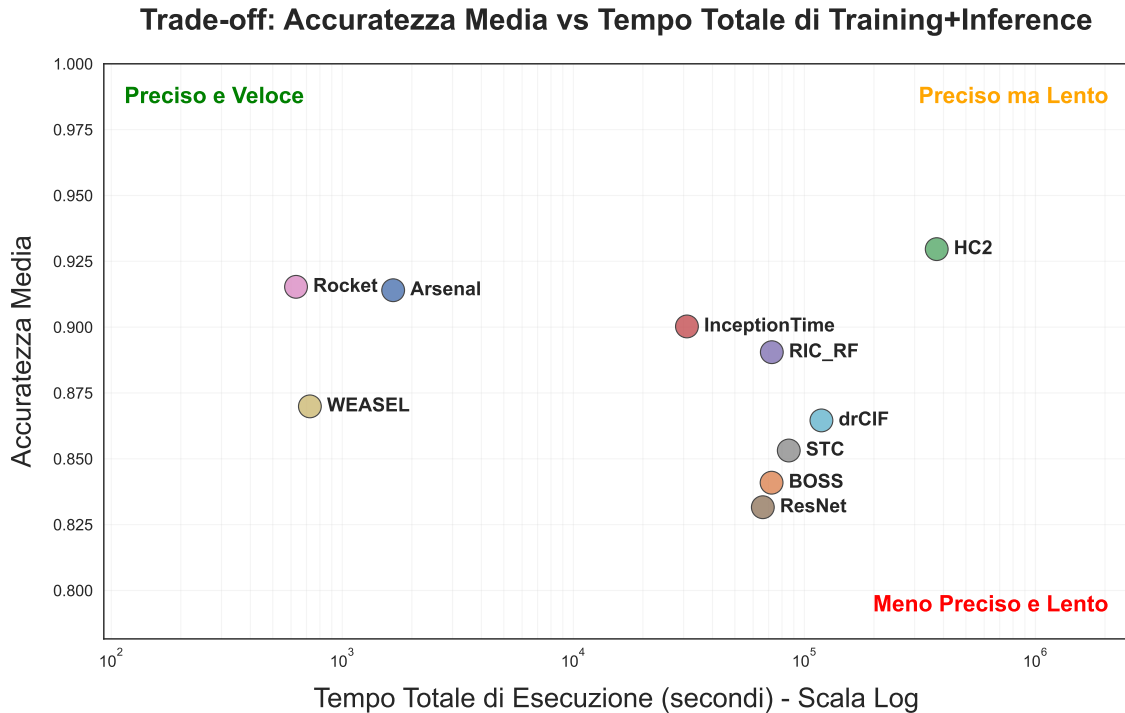


Figura 2.4: UCR Datasets: Accuratezza media vs tempi esecuzione totali.

Compromesso tra Accuratezza e Velocità. La Figura 2.4 offre una visione d'insieme del bilanciamento tra accuratezza e costi computazionali. HC2 si conferma il modello più accurato, ma anche il più lento del benchmark. ROCKET si distingue per essere il più veloce e il secondo più accurato, risultando più rapido di WEASEL e più preciso di Arsenal (il quale, tuttavia, è più lento di un ordine di grandezza).

Per quanto riguarda il Deep Learning, InceptionTime e ResNet mostrano tempi di esecuzione simili, ma il primo garantisce un netto vantaggio in termini di accuratezza. È interessante notare l'evoluzione dei modelli basati su dizionari: WEASEL rappresenta un salto generazionale rispetto a BOSS, superandolo sia in velocità che in precisione. BOSS registra performance simili a ResNet, restando però leggermente meno accurato di DrCIF e STC, pur essendo di poco più veloce nonostante i limiti temporali (*contract time*) imposti a questi ultimi due.

Infine, RIC, grazie alla sua struttura lineare e all'uso della Rotation Forest, presenta prestazioni paragonabili a quelle di InceptionTime, riuscendo a superare i più blasonati DrCIF e STC sia sotto il profilo dell'accuratezza che della velocità.

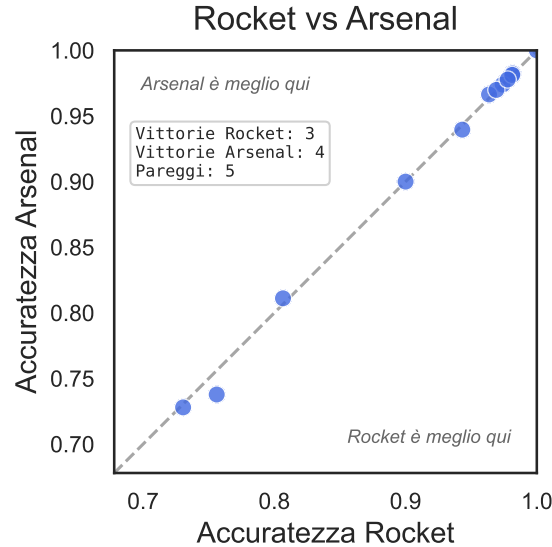


Figura 2.5: Scatter plot di Arsenal contro Rocket.

2.4.3 Analisi dei confronti diretti

La Figura 2.5 pone a confronto le prestazioni di ROCKET e Arsenal sui dataset analizzati. I risultati indicano che i due metodi sono statisticamente comparabili in termini di accuratezza: essi mostrano performance equivalenti su 5 dei 12 dataset e risultati pressoché identici nei restanti casi. Tale evidenza è ulteriormente confermata dal *Critical Difference (CD) diagram* (Figura 2.2), in cui il *rank* medio dei due algoritmi risulta quasi sovrapponibile.

Si può dunque affermare che il maggior costo computazionale richiesto da Arsenal non si traduca in un sensibile miglioramento dell'accuratezza rispetto alla versione singola di ROCKET. Tuttavia, all'interno dell'architettura di HIVE-COTE 2.0, Arsenal ricopre un ruolo fondamentale, infatti è stato progettato per sostituire ROCKET nel meta-ensemble, garantendo la stessa efficacia predittiva ma con il vantaggio cruciale di generare distribuzioni di probabilità tramite i voti dell'ensemble. Questa caratteristica è indispensabile per il controllore CAWPE, che necessita di tali probabilità per il calcolo dei pesi di ponderazione.

Attraverso la Figura 2.6 è possibile analizzare le prestazioni di HIVE-COTE 2.0 in relazione a tre delle sue quattro componenti fondamentali: Arsenal, STC e Dr-CIF. La superiorità di HC2 rispetto ai singoli moduli risulta evidente, confermando l'efficacia del suo approccio. Solo Arsenal dimostra una competitività paragonabile a quella del meta-ensemble. Sebbene il confronto non includa il modulo TDE (Temporal Dictionary Ensemble), i risultati confermano che la robustezza di HC2 non è riconducibile a un unico algoritmo, bensì alla sinergia dei quattro classificatori coordinati dal meccanismo di pesaggio CAWPE.

2.5 Conclusioni e Sviluppi Futuri

Il presente lavoro di tesi ha fornito un'analisi comparativa approfondita tra i principali algoritmi allo stato dell'arte per la classificazione di serie temporali (*Time Series Classification*), valutando modelli basati su dizionari, trasformazioni convo-

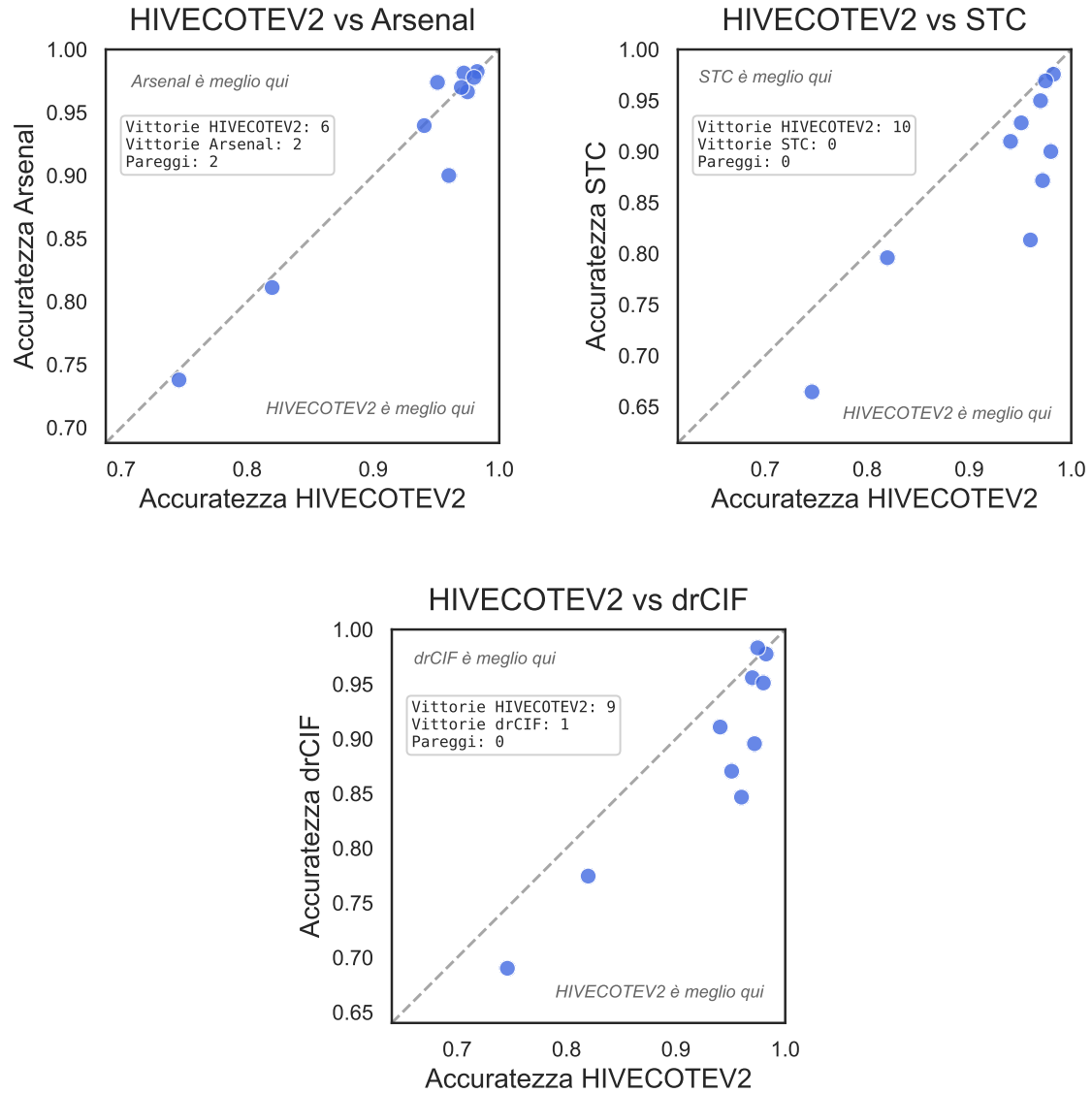


Figura 2.6: Scatter Plots dei confronti diretti tra HIVE-COTE 2.0 e 3 dei suoi 4 componenti interni (Arsenal, STC e DrCIF).

luzionali, intervalli e architetture di *Deep Learning*. Attraverso una sperimentazione condotta su 12 dataset eterogenei dell'archivio UCR, selezionati per rappresentare diversi scenari critici, è stato possibile delineare un quadro chiaro del compromesso tra accuratezza predittiva ed efficienza computazionale.

I risultati hanno confermato la superiorità di **HIVE-COTE 2.0** (HC2) in termini di accuratezza, consolidando la sua posizione come *gold standard* nel dominio della TSC. Tuttavia, l'analisi ha evidenziato come modelli quali **ROCKET** siano in grado di offrire prestazioni estremamente competitive con tempi di esecuzione ridotti di diversi ordini di grandezza. Sul fronte del *Deep Learning*, **InceptionTime** si è dimostrato un'architettura più solida ed efficiente rispetto alla classica *ResNet*, confermando l'efficacia dei moduli inception.

Lo studio ha fatto emergere i limiti strutturali di HC2, la cui complessità ha impedito l'analisi su dataset quali *ElectricDevices*, a causa dell'eccessivo consumo

di RAM, e *Fungi*, per l'insufficiente numero di campioni nel *training set*. L'impiego di HC2 su dataset con caratteristiche analoghe richiede, dunque, hardware ad alte prestazioni o tecniche di manipolazione dei dati non necessarie con altri metodi. D'altronde, HC2 è progettato per scenari in cui la massima accuratezza è prioritaria rispetto ai tempi di addestramento. Si può inoltre affermare che la ricerca estensiva dei parametri effettuata dalle componenti di HC2, non sia una condizione strettamente necessaria per ottenere prestazioni allo stato dell'arte, infatti l'imposizione di limiti temporali rigorosi produce comunque risultati mediamente superiori alla concorrenza. Al contrario, per scenari in cui il tempo di addestramento è un fattore critico, metodi rivoluzionari come **ROCKET** e **Arsenal** rappresentano la soluzione ideale, operando in una frazione del tempo richiesto da HC2 pur necessitando di un quantitativo di RAM non trascurabile.

In conclusione, l'analisi dimostra che non esiste un classificatore universalmente superiore in ogni contesto; la natura del problema specifico orienta la scelta dell'approccio. HC2 tenta di ovviare a questa eterogeneità integrando "esperti" provenienti da diversi domini, accettando come compromesso un incremento significativo dei costi computazionali.

Limitazioni. Nonostante i risultati ottenuti, lo studio presenta alcune limitazioni. In primo luogo, la dimensione del campione (12 dataset su 128 dell'archivio UCR), seppur rappresentativa di casi limite, potrebbe non riflettere appieno la varianza statistica necessaria per stabilire una gerarchia definitiva tra i modelli. Inoltre, i vincoli hardware e le stringenti limitazioni temporali imposte ad HC2 hanno parzialmente limitato l'esplorazione del potenziale del *meta-ensemble* e delle sue singole componenti.

Sviluppi Futuri. In ottica futura, la ricerca potrebbe espandersi su un numero maggiore di dataset e testare serie temporali multivariate o non stazionarie per testare la robustezza in contesti applicativi reali. Un ulteriore sviluppo potrebbe riguardare il *fine-tuning* dei parametri per ottimizzare l'accuratezza o l'efficienza in domini specifici, oltre alla valutazione della scalabilità degli algoritmi variando il grado di parallelizzazione (numero di CPU impiegate).

Bibliografia

- [1] Hassan Ismail Fawaz, Germain Forestier, Jonathan Weber, Lhassane Idoumghar, and Pierre-Alain Muller. Deep learning for time series classification: a review. *CoRR*, abs/1809.04356, 2018.
- [2] Patrick Schäfer. The boss is concerned with time series classification in the presence of noise. *International Journal of Data Mining and Knowledge Discovery*, 29(6), 2015.
- [3] Patrick Schäfer and Ulf Leser. Fast and accurate time series classification with WEASEL. *CoRR*, abs/1701.07681, 2017.
- [4] Hassan Ismail Fawaz, Benjamin Lucas, Germain Forestier, Charlotte Pelletier, Daniel F. Schmidt, Jonathan Weber, Geoffrey I. Webb, Lhassane Idoumghar, Pierre-Alain Muller, and François Petitjean. Inceptiontime: Finding alexnet for time series classification. *CoRR*, abs/1909.04939, 2019.
- [5] Anthony J. Bagnall, Jason Lines, Aaron Bostrom, James Large, and Eamonn J. Keogh. The great time series classification bake off: a review and experimental evaluation of recent algorithmic advances. *Data Min. Knowl. Discov.*, 31(3):606–660, 2017.
- [6] Angus Dempster, François Petitjean, and Geoffrey I. Webb. ROCKET: exceptionally fast and accurate time series classification using random convolutional kernels. *CoRR*, abs/1910.13051, 2019.
- [7] Matthew Middlehurst, James Large, Michael Flynn, Jason Lines, Aaron Bostrom, and Anthony J. Bagnall. HIVE-COTE 2.0: a new meta ensemble for time series classification. *CoRR*, abs/2104.07551, 2021.
- [8] Hoang Anh Dau, Eamonn Keogh, Kaveh Kamgar, Chin-Chia Michael Yeh, Yan Zhu, Shaghayegh Gharghabi, Chotirat Ann Ratanamahatana, Yanping, Bing Hu, Nurjahan Begum, Anthony Bagnall, Abdullah Mueen, Gustavo Batista, and Hexagon-ML. The ucr time series classification archive, October 2018.
- [9] Carl Henning Lubba, Sarab S. Sethi, Philip Knaute, Simon R. Schultz, Ben D. Fulcher, and Nick S. Jones. catch22: Canonical time-series characteristics. *CoRR*, abs/1901.10200, 2019.
- [10] J.J. Rodriguez, L.I. Kuncheva, and C.J. Alonso. Rotation forest: A new classifier ensemble method. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 28(10):1619–1630, 2006.

-
- [11] Jon Hills, Jason Lines, Edgaras Baranauskas, James Mapp, and Anthony Bagnall. Classification of time series by shapelet transformation. *Data Min. Knowl. Discov.*, 28(4):851–881, July 2014.
 - [12] Zhiguang Wang, Weizhong Yan, and Tim Oates. Time series classification from scratch with deep neural networks: A strong baseline. *CoRR*, abs/1611.06455, 2016.